

Git & GitHub

Practical Developer Workshop

 Version Control
CLI

 Collaboration



Learn by doing

Install tools, authenticate, configure identity, stage, commit, branch, merge, and recover safely.

1. Setup

Install Git and GitHub CLI with the least extra tooling possible.

2. Connect

Authenticate GitHub and configure your Git identity.

3. Practice

Use the daily commands developers run in real projects.

Edit

Stage

Commit

Share

>_ Windows

```
winget install --id Git.Git -e  
--source winget  
git --version
```

If winget is blocked, download Git for Windows from git-scm.com.

>_ macOS

```
xcode-select --install  
git --version
```

Alternative if Homebrew already exists:

```
brew install git
```

>_ Ubuntu/Debian

```
sudo apt update  
sudo apt install git -y  
git --version
```

Use your distro package manager for Fedora, Arch, or openSUSE.

>_ Windows

```
winget install --id GitHub.cli  
gh --version
```

>_ macOS

```
brew install gh  
gh --version
```

>_ Ubuntu/Debian

```
sudo apt update  
sudo apt install gh -y  
gh --version
```

If `gh` is old or unavailable, use the official GitHub CLI apt repository.

Why CLI first?

Terminal commands are copyable, repeatable, and work the same in local projects, servers, and CI.

>_ Interactive browser login

```
gh auth login
```

Choose these options during setup:

- GitHub.com
- HTTPS for Git operations
- Login with browser
- Authenticate Git operations: Yes

>_ Confirm the login

```
gh auth status
```

Good habit

Run authentication once per machine, then verify before a workshop or demo.

>_ Set global identity

```
git config --global user.name "Your Name"  
git config --global user.email "you@example.com"
```

>_ Check configuration

```
git config --global --list
```

What this controls

Every commit records an author name and email. Use the same email connected to your GitHub account when possible.

Per-project override

Inside one repository, remove `-global` to set identity only for that project.

>_ Daily loop

```
git status
git add .
git status
git commit -m "<commit_topic>" -m "<
    commit_description>"
```

How to read it

- git status: see changed, staged, and untracked files.
- git add .: stage current folder changes.
- git commit: save a named checkpoint locally.

>_ Fix the latest local commit

```
git add .
git commit --amend
```

>_ Create and inspect branches

```
git checkout -b <new_branch_name>  
git branch
```

>_ Delete a finished local branch

```
git branch -D <branch_to_be_deleted>
```

Branch naming examples

- feature/login-page
- fix/navbar-overlap
- docs/setup-guide

Rule of thumb

One branch = one focused change.

>_ Merge a completed branch

```
git checkout main  
git merge <branch_name>
```

>_ Abort a problematic merge

```
git merge --abort
```

Merge checklist

- Commit or stash your work first.
- Merge into the branch that should receive changes.
- If conflicts are confusing, abort and ask for review.

>_ Mini workflow

```
git checkout -b feature/practice
git status
git add .
git commit -m "Practice change" -m "Update files during
    Git workshop"
git checkout main
git merge feature/practice
git branch -D feature/practice
```

Remember

- Commands are real text in this PDF.
- Copy from the code boxes during practice.
- Start with `git status` whenever unsure.



Ship small, safe changes.